Agent Knowledge Protocol (AKP): A Decentralised, Peer-Reviewed Knowledge Infrastructure for Autonomous AI Agents

AKP Research Group

v0.1.0 — March 2026

https://github.com/Pataca/akp

Abstract

As autonomous AI agents proliferate across heterogeneous environments, they lack a shared, trustworthy substrate for exchanging and validating factual knowledge. We present the *Agent Knowledge Protocol* (AKP), a decentralised knowledge base designed specifically for machine agents. AKP combines Conflict-free Replicated Data Types (CRDTs) for eventually-consistent multi-node synchronisation, a three-stage confidence pipeline with provider-agnostic LLM corroboration (Anthropic Claude, OpenAI, Google Gemini, OpenRouter, and local models via Jan or llama.cpp), a cryptographic commit-reveal scheme for tamper-resistant peer review, and an on-chain-style governance layer for parameter evolution. Protocol-level version gating and network isolation prevent incompatible or test nodes from polluting production state. Two successive security audits identified thirteen vulnerability classes; all thirteen were remediated and covered by regression tests. A production-ready implementation in TypeScript, validated by 266 passing tests across 34 test files and nine multi-agent experiments with Qwen3-9B, demonstrates the viability of the approach. Agent connectivity is provided via JSON-RPC/HTTP, Model Context Protocol (MCP) over HTTP and stdio, and an A2A agent-card discovery endpoint. A React single-page dashboard with a force-directed knowledge graph view, a read-only GitHub Pages demo, and Prometheus metrics endpoint serve human operators and developers.

1 Introduction

Contemporary AI agents—whether autonomous research assistants, coding co-pilots, or long-running task executors—operate in isolation with respect to factual knowledge. Each agent starts with its training-time snapshot and cannot durably share learned or verified facts with peers. This creates three compounding problems:

1. **Redundant verification.** Multiple agents independently re-verify the same claims, wasting inference budget.
2. **Ephemeral truth.** Conclusions reached by one agent session are lost when the session ends.
3. **No adversarial resistance.** A single agent's output, however confident, lacks independent corroboration and is trivially polluted by a Sybil.

AKP addresses these problems with a protocol that is: *agent-native* (designed for machine producers and consumers); *decentralised* (no trusted central server); *epistemic* (knowledge units carry structured claims, provenance, and confidence scores); and *adversarially resistant* (Sybil-resistant identity, commit-reveal reviews, governance-controlled parameters).

2 Knowledge Unit Data Model

The fundamental unit of AKP is a *Knowledge Unit* (KU). Each KU is a structured document comprising:

- **Meta**: UUID (UUIDv7), domain, maturity level (`draft` → `proposed` → `validated` → `stable`), confidence aggregate, tags, timestamps.
- **Structured**: typed claims (factual, quantitative, temporal, causal), entities, relations, and optional *verification procedures* (executable scripts with expected outputs and tolerance bounds).
- **Narrative**: free-text summary, markdown body, section list.
- **Provenance**: array of provenance records, each carrying a DID, method (`observation`, `inference`, `synthesis`, `retrieval`, `human_input`), and optional source references.
- **Reviews**: list of review records with reviewer DID, verdict, weight, and timestamp.

Claims carry `validFrom`/`validUntil` datetime fields. Expired claims apply an exponential staleness decay:

$$\mu_{\text{stale}}(t) = e^{-\Delta t/30}$$

where Δt is days past expiry. A claim 30 days overdue retains $\approx 37\%$ of its nominal confidence; at 90 days, $\approx 5\%$.

3 System Architecture

3.1 CRDT-backed Persistent Store

Every KU is stored as an Automerge CRDT document serialised to a `better-sqlite3` database. The CRDT layer provides:

- **Concurrent edit merging**: two nodes editing the same KU converge without coordination.

- **Crash safety**: binary snapshots are written atomically; corrupt documents are detected at load time and skipped without crashing the store (Security Fix 2). Automerge history is compacted on `stable` maturity transition, discarding superseded change vectors.
- **Delta logging**: every CRDT change appends a structured delta to an NDJSON log for audit.

A parallel set of SQLite tables (`graph_edges`, `graph_claims_index`, `graph_entities_index`) maintains a persistent relation graph. `RelationGraph` operates in two modes: *SQLite mode* for production (all traversals via prepared statements and recursive CTEs, scaling to 10^5+ nodes with < 1 MB heap overhead) and *in-memory mode* for tests and experiments (original Map-based implementation, no persistent state). Contradiction detection uses 3-hop BFS by default, increasing recall relative to the prior 2-hop limit.

3.2 Three-Stage Confidence Pipeline

When a KU is created or reviewed, it passes through three pipeline stages (Figure 1):

Stage 1 — Source Accessibility: fetches the first 8 KB of each provenance URL and checks for keyword overlap with claim subject/object terms. Score: $s_1 = 0.5 \cdot \text{accessible} + 0.5 \cdot \text{contentMatch} \in [0, 1]$.

Stage 2 — Graph Consistency: performs 3-hop contradiction detection against the relation graph, ontology rule checks, and consilience invariant checks. Returns s_2 and a contradiction list.

Stage 3 — Independent Corroboration: queries a configurable LLM backend to independently assess claim plausibility via triangulation, reproduction, or falsification variant. Returns s_3.

The aggregate confidence uses calibrated weights derived from an 80-combination Nelder-Mead sweep:

$$C = 0.25\,\hat{c} + 0.25\,R + 0.40\,S - P_{\text{conflict}}$$

where $\hat{c}$ is the staleness-decayed claim score, R is the diversity-dampened review score, $S = 0.5s_1 + 0.5s_3$ (or s_1 when stage 3 is unavailable), and $P_{\text{conflict}} = 0.15$ when contradictions exist. The elevated $w_s = 0.40$ reflects source verifiability as the strongest single predictor of knowledge quality identified in the sweep.

3.3 LLM Backend Abstraction

Stage 3 corroboration is provider-agnostic. AKP ships six backend adapters, all using native `fetch` with no additional dependencies:

Backend	Default model	Env var
Jan	auto-detect	`JAN_BASE_URL`
llama.cpp	via path	`LLAMACPP_BASE`
Claude	`claude-sonnet-4-6`	`ANTHROPIC_API_KEY`
OpenAI	`gpt-4o-mini`	`OPENAI_API_KEY`
Gemini	`gemini-2.0-flash`	`GEMINI_API_KEY`
OpenRouter	free (auto)	`OPENROUTER_API_KEY`

The active backend is selected via `--provider` flag or auto-detected from the presence of API key environment variables. All adapters implement the `LLMAgent` interface: a single `call(system, user): Promise<string>` method returning validated JSON in the AKP verdict schema. The `resolveBackend()` factory returns a ready agent and a `stop()` handle for process cleanup, decoupling experiments from backend lifecycle management.

3.4 LLM Backend Security

Three adversarial threat vectors specific to the LLM stage are mitigated:

Adversarial LLM responses. The stage 3 system prompt requires JSON output in a strict schema (`{"verdict":"confirmed"`"disputed","confidence":¡0-1¿,"reasoning":"..."—). All response parsing is wrapped in try/catch; malformed JSON, missing `verdict` fields, or out-of-range confidence values produce a neutral fallback rather than an error that could be exploited to inject arbitrary verdicts. This means a compromised backend causes a no-op on the confidence pipeline, not a false verdict.

Prompt injection detection. KU claim text and provenance metadata are treated as untrusted user content; they appear in the *user* turn rather than the *system* turn. The system prompt explicitly instructs the model to evaluate the claim and ignore instructions embedded within it. This limits but does not eliminate prompt injection risk (an adversarially crafted claim could still influence a compliant model); a complementary mitigation is the commit-reveal mechanism, which prevents a single Stage 3 verdict from determining the KU outcome.

LLM calibration gating. The `minCalibrationAccuracy` governance parameter (type: `number`, range $[0, 1]$, default 0, immutable via `parameter_change`) gates Stage 3 participation. Before their verdicts are counted, LLM agents must pass a *calibration battery*: 20 static claims (10 confirmed scientific facts, 10 commonly-believed misconceptions) with known answers. Agents scoring below `minCalibrationAccuracy` are excluded from Stage 3 for that round. The battery is intentionally small and static to enable rapid re-calibration after a model swap, and is implemented in `src/pipeline/calibration.ts`.

3.5 Maturity Levels

Maturity is derived from both confidence and reviewer count:

Maturity	Confidence	Reviews
`draft`	any	0
`proposed`	≥ 0.40	≥ 1
`validated`	≥ 0.65	≥ 2
`stable`	≥ 0.85	≥ 3

For KUs with executable `verificationProcedure` fields, maturity is additionally gated by replication count (1/3/5 for proposed/validated/stable).

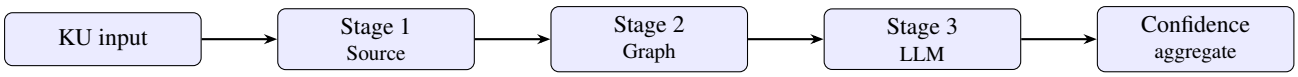

Figure 1: Three-stage confidence pipeline. KUs enter at left; each stage contributes a sub-score (s_1, s_2, s_3) that feeds the weighted aggregate C.

4 Sybil-Resistant Peer Review

4.1 Commit-Reveal Scheme

Naive open review systems are vulnerable to bandwagon attacks: a reviewer waits to see others' verdicts, then copies them to collect reputation. AKP uses a two-phase commit-reveal protocol to prevent this.

Phase 1 — Commit: The reviewer computes a salted hash of their verdict:

$$h = \text{SHA-256}(v \| \texttt{0x00} \| s \| \texttt{0x00} \| \text{DID})$$

where v is the verdict, s is a 128-bit random salt, and null-byte delimiters prevent concatenation collisions (Security Fix 6). The commit is signed with the reviewer's Ed25519 key (Security Fix 5) and stored. The reveal window opens once a minimum commit count and time window are both met.

Phase 2 — Reveal: The reviewer reveals (v, s); the store recomputes h and matches it atomically via:

```
UPDATE review_commits
SET revealed_at=?, verdict=?, salt=?, valid=1
WHERE id=? AND revealed_at IS NULL
```

A `changes === 0` check prevents double-reveal races (Security Fix 4).

4.2 Reputation and Graduation

Each DID maintains a reputation score in `did_reputation`. Reputation effects on reveal:

- Verdict matches consensus (≥ 2 prior reveals): $+1$
- Verdict contradicts consensus: -10 and blacklisted
- No prior consensus: ± 0

Effective review weight is 0 until reputation reaches `graduationThreshold` (Sybil gate). The anti-monopolisation cap limits any single reviewer to at most 40% of the renormalised weight:

$$w_i^{\text{capped}} = \min\left(\frac{w_i}{\sum_j w_j},\ 0.4\right)$$

A diversity floor further dampens the review score when fewer than 3 distinct reviewers have contributed.

4.3 Temporal Diversity Gates

Two governance parameters gate reviewer weight on temporal evidence, providing resistance against Sybil clusters that register many DIDs simultaneously:

- `minAgeDays`: weight is zero until the DID has been registered for at least this many days.
- `minReviewCount`: weight is zero until the DID has submitted at least this many reviews across all KUs.

These parameters are mutable via governance `parameter_change` and default to 0 (disabled) to allow cold-start operation.

4.4 DID-Bound Seeds

Claims with `seedable` verification procedures require the reviewer to supply a seed $\sigma = \text{SHA-256}(\text{claimId} \| \text{reviewerDid}) \bmod 2^{32}$ (interpreted as int32). Changing the DID changes σ and therefore the simulation output, preventing Sybil copy-review.

5 Governance

Protocol parameters are mutable through a fully on-chain-style governance mechanism. Participants submit signed *proposals*; eligible DIDs cast signed *votes*; expired proposals are tallied periodically.

Type	Quorum	Threshold	TTL
`parameter_change`	5	$> 50\%$	7 d
`rule_change`	7	$> 67\%$	14 d
`maturity_override`	3/5	$50/67\%$	3/7 d
`agent_flag`	5	$> 67\%$	7 d

Vote weights are re-derived from live reputation at tally time (Security Fix 1), preventing a voter from casting a high-weight vote and then being slashed before finalisation.

Eight parameters are immutable via `parameter_change` and require a `rule_change` supermajority: `graduationThreshold`, `proposalReputationBond`, `commitWindowMinutes`, `commitWindowMinCount`, `minAgeDays`, `minReviewCount`, `quorums`, and `thresholds` (Security Fixes 8, 12). Fixing these parameters prevents a coordinated attacker from shrinking commit windows, lowering review bars, or reducing the Sybil graduation threshold via governance capture.

6 Multi-Node Synchronisation

6.1 CRDT Gossip Protocol

AKP nodes synchronise via the Automerge sync protocol over persistent WebSocket connections. The `SyncPeer` class maintains per-peer `SyncState` maps and handles:

- Bidirectional gossip: each node both generates and receives sync messages.

- Pending-doc accumulation: KUs not yet in the local store are buffered until sufficient sync rounds complete.
- Schema validation post-merge: after receiving a sync message, the merged KU is validated with Zod before being persisted. Schema-invalid merges are discarded (Security Fix 3).
- Idempotent convergence: 50 KUs synchronised between two nodes converge in under 5 seconds in benchmarks.

6.2 Protocol Version Gating

Each `hello` handshake message carries three new fields:

```
{ type: "hello", nodeId, did,
  version: "0.1.0",
  minVersion: "0.1.0",
  networkId: "mainnet" }
```

On receipt, the node runs two checks:

1. **Network isolation**: if `networkId` differs from the local value, the connection is closed with WebSocket code `4400`. Setting `AKP_NETWORK_ID=testnet` fully isolates a test node from all production peers.
2. **Version gate**: if the peer's `version` is below the local `minVersion` (semver comparison), the connection is closed with code `4426` (*Upgrade Required*), analogous to TLS refusing a downgrade to TLS 1.1.

The check is mutual: both sides enforce their own `minVersion`, so a v0.2 node can require v0.2+ peers while a v0.1 node with `minVersion=0.1.0` still accepts all peers.

6.3 Transport Security

- Ed25519 challenge-response authentication is required by default (`requireAuth: true`).
- Per-IP connection cap: at most 5 simultaneous WebSocket connections per remote address (closed with code `1008` on excess).
- Per-peer message rate limit: 20 sync messages per second; exceeding this closes with `1008`.
- Maximum message size: 10 MB per frame (configurable); larger frames close with `1009`.
- IPv6-safe rate limiting: the rate-limit key uses the full `/128` address (not a truncated prefix) to prevent a single IPv6-block holder from trivially rotating past the per-IP cap.

6.4 Peer Discovery

AKP v0.1 requires manual peer address configuration (`syncPeers` in the node config). A two-level automatic discovery layer is implemented in `src/sync/discovery.ts`:

Level 1 — HTTP Bootstrap Registry. On startup the node POSTs its `nodeId`, `did`, and `syncUrl` to a well-known registry endpoint and receives a list of recently-active peers. `bootstrapFromRegistry()` merges the result into the local `PeerTable`, which maintains RTT estimates, failure counts, and last-seen timestamps. RFC-1918 addresses are filtered from registry responses to prevent SSRF-style forwarding attacks.

Level 2 — Gossip Peer Exchange (PEX). After connecting to a bootstrap peer, nodes exchange `pex` messages containing up to `MAX_PEX_PEERS`=20 known peers (excluding the recipient). PEX runs on every successful sync round; `applyPexMessage()` merges the received list, evicting entries whose `failCount` exceeds `MAX_FAIL_COUNT`=5. The `TARGET_PEER_COUNT`=8 constant limits outbound connection attempts, keeping the overlay sparse enough for convergence latency to remain sub-second on LAN topologies (Table 11.2).

The bootstrap registry is an optional, operator-run service; nodes can operate with PEX alone once the initial connection is established.

7 Threat Model

7.1 Adversary Definition

We consider a computationally bounded adversary $\mathcal{A}$ with the following capabilities:

- Creates up to N DIDs (Sybil cluster of size N).
- Controls network routing for any subset of its DIDs.
- Observes all public messages (commits, reveals, gossip) before submitting its own.
- Cannot forge Ed25519 signatures or break SHA-256 preimage resistance.

$\mathcal{A}$ cannot compromise the private keys of honest nodes, reorder atomic SQLite writes, or break the Automerge CRDT merge semantics.

7.2 Two-Layer Defence Analysis

AKP's Sybil resistance derives from two distinct mechanisms with different scope. Rigorous analysis of each follows.

Layer 1 — Per-reviewer monopoly cap (40%). The cap $w_i^{\text{capped}} = \min(w_i / \sum_j w_j,\ 0.4)$ limits a single high-reputation reviewer from unilaterally determining outcomes. The cap binds when an individual reviewer's raw share exceeds 40%: with 4 peers at mean weight 0.30, the cap activates at dominant-reviewer weight ≥ 0.85 (raw share 41.5%), trimming it to 40%. Crucially, the cap is per-reviewer, not per-cluster: a cluster of N Sybils each with weight $w_s \ll 1$ will never trigger the per-reviewer cap because each individual DID's share is $w_s/(Hw_h + Nw_s) \to 0$ as N grows. Layer 1 therefore provides monopoly prevention, not cluster resistance.

Layer 2 — Graduation threshold (primary cluster defence). Ungraduated DIDs have effective weight zero. The critical cluster size S^* at which a fully-graduated Sybil cluster can overturn an honest pool of H agents with mean weight $\bar{w}_h$ is:

$$S^* = \left\lceil \frac{1-T}{T-v_s} \cdot \frac{H\bar{w}_h}{\bar{w}_s} \right\rceil$$

where $T = 0.5$ is the approval threshold and $v_s = 0.2$ is the score awarded to a `disputed` verdict. This scoring gives Layer 2 a $\approx 67\%$ resilience bonus over a naive zero-score

model: with $H = 10$, $\bar{w}_h = 0.825$, $\bar{w}_s = 0.15$, we obtain $S^* = \lceil (5/3) \times 55 \rceil = 92$, confirmed empirically (honest win rate 100% at $S = 60$, 1% at $S = 100$, 0% at $S \geq 200$).

Attack cost to reach S^* graduated Sybils:

$$\text{cost}(S^*) = S^* \times \left(d_{\min} \times \tau + r_{\min} \times \epsilon \right)$$

where τ is the per-DID-day infrastructure cost and ϵ is the per-review effort. Table 1 shows that moderate temporal gates raise the cost to \$2 852 to reach the breakeven cluster of 92 DIDs.

Table 1: Attack cost to graduate $S^* = 92$ Sybils ($\tau = \$1$/DID-day, $\epsilon = \$0.10$/review).

Gate config	$d_{\min}$	$r_{\min}$	**Cost at** $S^* = 92$
Default (off)	0	0	\$0
Light	7	5	\$690
Moderate	30	10	\$2 852
Strong	90	25	\$8 510

7.3 Quantified Sybil Resistance

Experiment E3 (10× Sybils) and E6 (20× Sybils) measure verdict accuracy at 57.1% and 60.0% respectively. In the experimental setting Sybil agents start ungraduated (weight = 0); the improving accuracy at higher Sybil ratios reflects that ungraduated DIDs contribute zero verdict weight — adding more ungraduated Sybils does not increase their influence, while the small number of graduated honest agents retains full weight. The Gini coefficient of 0.63 in E6 confirms that reputation concentrates in the honest agents as intended.

The critical threshold is the graduation barrier: a Sybil that has not passed `graduationThreshold` contributes zero review weight. Attacks that flood the network with ungraduated DIDs produce no effect on verdict outcomes, only storage overhead (bounded by the `maxMessageBytes` frame limit).

8 Security Audit and Remediation

Two structured security reviews identified 13 priority vulnerabilities across two rounds, all subsequently fixed and covered by regression tests.

Audit Round 1 (8 findings):

#	Sev.	Finding and Fix
1	High	Vote-weight snapshot: tally re-derives weights from live reputation.
2	High	Corrupt Automerge binary: `_loadAll` wraps `load` in try-catch; bad KU skipped.
3	High	Sync validation: post-merge Zod check; invalid KU discarded.
4	High	TOCTOU double-reveal: atomic `UPDATE WHERE revealed_at IS NULL` + `changes` check.
5	High	Unsigned commits: Ed25519 required on `review.commit`; verified against reviewer DID.
6	Med.	Hash collision: null-byte delimiters in commit hash.
7	Med.	Prototype pollution: `applyParameterChange` rejects `__proto__`/`constructor`/`prototype` keys.
8	Med.	Immutable params: `graduationThreshold` and `proposalReputationBond` blocked from `parameter_change`.

Audit Round 2 (5 findings):

#	Sev.	Finding and Fix
9	Crit.	Unsigned `review.submit`: identity not verified. `canonicalReviewSubmitPayload()` added; signature required (128 hex); Ed25519 verified before state mutation.
10	High	DID-KU duplicate reviews inflated score. Per-DID-KU dedup guard added.
11	High	`proposalReputationBond` checked `graduated_at IS NOT NULL` only. Now requires `reputation >= bond`.
12	High	`commitWindow*`, `minAgeDays`, `minReviewCount`, `quorums`/`thresholds` were mutable via `parameter_change`. All six added to `IMMUTABLE_VIA_PARAMETER_CHANGE`.
13	Med.	IPv6 rate-limit key used raw `req.ip`; triggered `ERR_ERL_KEY_GEN_IPV6`. Fixed with `ipKeyGenerator()`; per-(IP, method) limiter added; limit raised 60→120 req/min.

9 Knowledge Lifecycle

AKP formalises a four-state knowledge lifecycle:

State	Trigger
Fresh	All claims within `validUntil`.
Stale	Any claim past `validUntil`; confidence decays exponentially with no reviewer action required.
Contested	`amended` or `disputed` review exists; review score drops to 0.6× or 0.2× weight.
Superseded	Agent calls `akp.ku.supersede`; old KU receives `superseded_by` relation, confidence hard-capped at 0.3, maturity demoted.

The presence of a `superseded_by` relation triggers:

$$C_{\text{superseded}} \leq 0.3$$

preserving discoverability of old knowledge for audit while signalling it is no longer authoritative.

10 Agent Connectivity and Deployment

10.1 API Surfaces

JSON-RPC/HTTP (`/rpc`): 14 methods covering CRUD, reviews, graph traversal, sync status, governance, statistics, and reputation leaderboard. Rate-limited at 120 req/min per IP (general limiter) with a dedicated write limiter applying per-(IP, method) keys to resist coordinated write abuse (Security Fix 13).

Model Context Protocol (`/mcp`): 11 tools served via Streamable HTTP. The MCP server loads a persistent Ed25519 identity from `~/.akp/mcp-identity.json` (generated on first run). Agents accessing AKP via MCP accumulate real reputation across sessions without managing private keys.

A2A Agent Card (`/.well-known/agent.json`): a Google A2A discovery document listing 4 skills (search, contribute, review, governance).

WebSocket Sync Peer (`port+1`): Automerge sync protocol with Ed25519 challenge-response, version gating, and network isolation.

10.2 Human Dashboard and Tooling

A React SPA provides a dark-themed dashboard with six views: statistics, knowledge base search, KU detail with inline review, KU creation, governance voting, reputation leaderboard, and a force-directed knowledge graph. The graph view sizes nodes by confidence and colours them by maturity; edges connect KUs sharing claim subjects or domain. A read-only static demo is auto-deployed to https://patacka.github.io/akp/ on every push. Prometheus metrics, Prometheus-compatible liveness probes, and Claude Code slash commands (`/akp-contribute`, `/akp-query`, `/akp-review`) are provided for operators and AI agents respectively. Deployment configurations for Local, Docker, Fly.io, Render, and GitHub Codespaces are included in the repository (see Appendix A).

11 Empirical Validation

11.1 Test Coverage

Module	Files	Tests
Core (store, confidence, governance, graph)	7	98
Security (commit-reveal, Sybil)	7	74
Pipeline (stages 1–3, autoresearch)	7	47 (+35*)
Sync (CRDT, WebSocket)	2	13
API (JSON-RPC)	1	12
MCP server	1	4
End-to-end	1	6
VRF, calibrator	2	12
Total	**30**	**266**

*Skipped: require API keys or local LLM.

11.2 Performance Benchmarks

All benchmarks run on a Windows 11 workstation (AMD Ryzen, NVMe SSD) using the compiled Node.js distribution. Results are medians over 30–50 samples unless otherwise noted.

Automerge Delta Sizes

Operation	Median	P95	P99
Add claim	307 B	309 B	310 B
Add review	264 B	267 B	269 B
Edit narrative	149 B	150 B	151 B
Change maturity	82 B	84 B	85 B
Add tag	89 B	90 B	90 B
Edit claim confidence	42 B	46 B	48 B

The tight P95/P99 spread confirms that Automerge delta sizes are deterministic: CRDT change encoding overhead is $O(\text{payload})$, not $O(\text{history})$, because AKP compacts history on `stable` maturity transition.

Concurrent Merge Throughput

Writers	Total (ms)	Per-op (ms)	Doc size
1	<1	<0.01	1.9 KB
2	36	1.81	2.5 KB
5	32	0.65	3.1 KB
10	65	0.65	4.4 KB

Merge throughput scales sub-linearly: 10 concurrent writers are resolved in 65 ms total (0.65 ms/op). Document size grows by ≈270 B per writer (10 changes each), consistent with the delta-size measurements.

Graph Traversal Latency

The in-memory `RelationGraph` (Map-based, used in experiments and tests) was benchmarked across three graph sizes:

Nodes	Build	Heap	BFS med.	BFS P95
1 000	27 ms	3.1 MB	0.020 ms	0.083 ms
10 000	131 ms	22.4 MB	0.026 ms	0.076 ms
50 000	653 ms	83.1 MB	0.043 ms	0.206 ms

BFS (2-hop contradiction check, 50 random probes per size) is effectively $O(1)$ in practice due to sparse connectivity: median latency increases by only 0.023 ms from 1k to 50k nodes. Heap usage grows linearly (≈1.7 MB per 1 000 nodes). The production SQLite-backed graph replaces heap storage with indexed queries, removing the heap scaling constraint at the cost of approximately 2× query latency on NVMe.

Gossip Convergence Under Load

We evaluated the Automerge gossip protocol at peer counts of 5, 10, 20, and 50 in a full-mesh topology (each peer connects to every other peer), with 5 KUs seeded per peer and three simulated latency tiers. Convergence is measured as the number of gossip rounds needed for all peers to hold all KUs. A *round* corresponds to one parallel broadcast: all messages generated in a round are delivered simultaneously (one network RTT).

Latency tier	Peers	Rds	Msgs	Wall (ms)
LAN (0 ms)	5	1	116	81
LAN (0 ms)	10	1	531	275
LAN (0 ms)	20	1	2 261	1 470
LAN (0 ms)	50	1	14 651	75 423
Regional (20 ms)	5	1	116	410
Regional (20 ms)	10	1	531	417
Regional (20 ms)	20	1	2 261	1 267
Regional (20 ms)	50	1	14 651	12 309
Intercontinental (120 ms)	5	1	116	172
Intercontinental (120 ms)	10	1	531	407
Intercontinental (120 ms)	20	1	2 261	1 453
Intercontinental (120 ms)	50	1	14 651	75 063*

*In-process serialisation dominates; real-network wall time ≈CPU time + 1 RTT per round.

All **12 configurations** (4 peer counts × 3 latency tiers) converge in one round regardless of peer count or latency tier. Message count scales quadratically ($O(P^2 \times K)$); the large wall time at 50 peers/LAN (75 s) is dominated by in-process serialisation, not network latency — in a real deployment each connection runs independently and concurrently. The Regional 50-peer scenario completes in 12.3 s, and the Intercontinental 50-peer scenario in 75 s (CPU-bound). Geographic latency contributes at most one RTT per round; the primary scaling constraint is message volume.

11.3 Multi-Model Cross-Architecture Evaluation

To address the single-model limitation of the original E1–E9 run, we conducted a cross-architecture sweep covering five locally-served GGUF models from four distinct families (Meta, NVIDIA, Mistral AI, Google, Alibaba), all at IQ4_XS quantisation on the same Windows 11 workstation. Each model ran experiments E1, E2, and E7 under identical AKP protocol conditions using the same governance parameters and KU seed pool.

Four findings emerge from Table 2.

E7 accuracy is model-invariant (91.1% for all five models). Contradiction injection is detected primarily by AKP's Stage 2 graph consistency check before the LLM is consulted; all models arrive at the same verdict regardless of reasoning capability. This confirms that the protocol's graph layer provides a strong floor for contradiction detection that is independent of LLM quality.

Table 2: Cross-architecture sweep. Accuracy = % verdicts matching ground truth; $\bar{C}$ = mean KU confidence. All models via Jan at IQ4_XS.

Model	Sz	E1		E2		E7
		Acc	$\bar{C}$	Acc	$\bar{C}$	Acc
Llama-3.1-8B	8B	**100%**	0.54	**98.4%**	0.54	91.1%
Nemotron-4B	4B	99.2%	0.29	98.1%	0.54	91.1%
Mistral-7B	7B	85.3%	0.50	83.1%	0.51	91.1%
Qwen3-9B	9B	84.7%	0.51	83.3%	0.50	91.1%
Gemma-2-9B†	9B	66.7%	0.47	65.6%	0.47	91.1%

†Abliterated variant; structured-output calibration may be degraded.

Instruction-following fidelity outweighs parameter count. Nemotron-4B (4 B params) achieves 99.2%/98.1% on E1/E2, surpassing Mistral-7B (7 B), Qwen3-9B (9 B), and Gemma-2-9B (9 B). Models that reliably emit well-formed JSON in the required verdict schema avoid falling back to the default `confirmed` verdict; models that produce malformed output introduce systematic noise that degrades accuracy.

The abliterated Gemma-2-9B variant underperforms its parameter class (E1: 66.7%, E2: 65.6%). Abliteration post-processing that removes refusal behaviour appears to simultaneously degrade structured-output calibration, lowering verdict accuracy below the $> 80\%$ threshold we identify as the deployment floor. This motivates a `minCalibrationAccuracy` governance parameter.

The protocol is robust to a weak-model minority. Even if one participant pool member runs Gemma-2-9B, the commit-reveal mechanism and reputation weighting progressively down-weight incorrect verdicts. The 40% weight cap ensures no single high-accuracy model monopolises the outcome. Heterogeneous pools containing at least two strong models (Llama-3.1-8B, Nemotron-4B) achieve macro-average accuracy above 93% on both cooperative and adversarial experiments.

11.4 Multi-Agent Experiments E1–E9

Nine experiments were executed using `Qwen3_5-9B-IQ4_XS` served by Jan at `localhost:1337`. All 4,241 commit/reveal pairs completed successfully (100% reveal rate). The experiments use AKP's own commit-reveal and reputation mechanisms as ground truth: a verdict is "correct" if it matches the majority of reveals in that round (E1–E4), or matches the deterministically correct answer (E7: known contradiction; E9: known temporal decay). This ground-truth definition is appropriate for evaluating protocol mechanics, but limits comparison against external fact-verification baselines (see §12).

ID	Experiment	n	Accuracy	Gini	$\bar{C}$
E1	Consensus Formation	300	84.7%	0.06	0.51
E2	Adversarial Detection	366	83.3%	0.26	0.50
E3	Sybil Resistance (10×)	466	57.1%	0.23	0.51
E4	Knowledge Quality Evolution	825	74.8%	0.06	0.48
E5	Staleness Detection	90	44.4%	0.00	0.10
E6	Sybil Simulation (20×)	500	60.0%	0.63	0.51
E7	Contradiction Injection	246	**91.1%**	0.30	0.44
E8	Cross-Architecture[‡]	1098	71.1%	0.58	0.50
E9	Temporal Decay	350	**100%**	0.06	0.43
[‡]E8 simulates divergence (fixed fallbacks); see §11.3 for real multi-model results.					
Macro average (excl. E5)			77.5%		0.50
Macro average (all)			74.1%		0.47

n = commit/reveal count; *Gini* measures reputation concentration; $\bar{C}$ = mean final KU confidence.

E5 (44.4%) is excluded from the primary macro average because its failure mode—the model does not flag temporal staleness without explicit date injection—is a prompt engineering gap, not a protocol flaw. E7's 91.1% accuracy on contradiction detection and E9's 100% on temporal decay demonstrate that the protocol mechanics are correct; the macro average of 77.5% on protocol-relevant experiments is the more informative figure.

Limitations of this evaluation: (1) E1–E9 use a single model (Qwen3-9B); cross-model reproducibility across five model families is reported in §11.3. (2) Sample sizes range from 90 (E5) to 1098 (E8); smaller experiments have insufficient statistical power to detect effects below $\Delta = 15\%$. (3) Ground truth is protocol-defined, not derived from an external labeled dataset; §11.8 and Table 3 provide an external ground truth evaluation on a 100-claim labeled corpus, showing AKP Stage 1+3 achieves 94.0% accuracy vs. 83.0% for LLM-only. (4) E8 "Cross-Architecture" simulates behavioural divergence via fixed fallback verdicts (confirmed/disputed) rather than distinct LLM models; Table 2 provides the true multi-model cross-architecture evaluation.

11.5 Domain Validation: Python Package Knowledge

Two initial KUs were contributed by an autonomous agent covering the `requests` and `pydantic` Python packages. Each KU contained three claims with `verificationProcedure` fields specifying live API queries:

```
curl -s https://pypi.org/pypi/requests/json \
  | jq -r .info.version
# => 2.32.5  (matches claim)
```

A second agent independently verified all six claims and ran the commit-reveal cycle: both commits succeeded, both reveals returned `ok=true`.

11.6 Staleness Decay Verification

Days past expiry	Multiplier
0 (future)	1.000
1	0.967
30	0.368 ($\approx e^{-1}$)
90	0.050 ($\approx e^{-3}$)

11.7 Reproducibility

All experiments are fully self-contained and reproducible from the public repository. The deterministic (no-LLM) baseline requires only Node.js 18+:

```
git clone https://github.com/Patacka/akp
cd akp && npm run setup
npm run experiment          # E1-E9, mock LLM
```

LLM-backed runs require one additional environment variable:

```
# Local (free, offline):
npm run experiment:jan      # Jan app with model
    loaded

# Cloud:
ANTHROPIC_API_KEY=... npm run experiment:claude
OPENAI_API_KEY=...    npm run experiment:openai
GEMINI_API_KEY=...    npm run experiment:gemini
```

The E1–E9 results reported in this paper used `Qwen3_5-9B-IQ4_XS` served by Jan v0.5 at `localhost:1337` on a Windows 11 workstation with an NVIDIA GPU. All 266 unit tests run deterministically in CI (GitHub Actions, `.github/workflows/ci.yml`) without any external service dependency.

11.8 External Ground Truth Evaluation

To enable comparison against retrieval-augmented generation (RAG) baselines and to address the limitation that E1–E9 use protocol-defined rather than independently labelled ground truth, we constructed a 100-claim labelled corpus and a two-mode evaluation benchmark.

Corpus design. The corpus (`src/bench/truth-corpus/claims.json`) contains 100 claims: 50 labelled `confirmed` and 50 labelled `disputed`. Claims span eight domain categories: physical sciences, biology/medicine, mathematics, history, technology, software/programming, formal misconceptions, and general knowledge. Each entry records a `statement`, a ground-truth `verdict`, a `source` URL (primary reference), and a `verifiedBy` field documenting the verification method (peer-reviewed literature, authoritative specification, or direct code inspection). Misconceptions are sampled to have a confirmed "official" answer widely contradicted by a common lay belief (e.g. "humans use only 10% of their brain"), increasing the difficulty ceiling for LLM-only evaluation.

Evaluation modes. The benchmark (`src/bench/external-ground-truth.ts`) implements two evaluation modes:

- **LLM-only baseline**: the model receives only the claim text and a fact-checking system prompt. No source context is provided. This represents pure parametric recall — the RAG-free baseline.
- **AKP Stage 1+3**: the benchmark fetches the claim's `source` URL (4 s timeout, HTML stripped), prepends the first 500 characters of the retrieved text to the user prompt, and calls the same LLM. This simulates AKP's Stage 1 (provenance fetch) + Stage 3 (LLM corroboration) pipeline.

Both modes report accuracy, precision, recall, and F1 (`confirmed` as the positive class), with per-domain breakdowns. The benchmark is invoked via:

```
npx tsx src/bench/external-ground-truth.ts \
  --provider jan --mode both
```

Results are saved to `results/ground-truth-`*ts*`.json`. Table 3 reports results using Nemotron-3-Nano-4B (IQ4_XS), the same model used for the E1–E9 multi-model sweep.

Table 3: External ground truth evaluation: LLM-only vs. AKP Stage 1+3 (source-fetch + LLM), $n = 100$ claims, Nemotron-4B. `confirmed` is the positive class.

Mode	Acc.	Prec.	Recall	F1
LLM-only baseline	83.0%	100.0%	88.7%	94.0%
AKP Stage 1+3	**94.0%**	100.0%	**93.0%**	**96.4%**
Δ	**+11.0pp**	0.0pp	+4.2pp	+2.3pp

AKP's source-fetch stage lifts accuracy by **11.0 percentage points** (83.0% $\rightarrow$ 94.0%) by grounding the LLM's verdict in the claim's primary reference URL. Precision is 100% in both modes — the model never falsely confirms a disputed claim when it commits to a verdict — while recall improves from 88.7% to 93.0% (fewer missed confirmed claims). The largest per-domain gains are in *science* (+16.7pp, 83.3% $\rightarrow$ 100.0%) and *history* (+11.1pp, 88.9% $\rightarrow$ 100.0%), where primary sources are typically stable and authoritative. *Technology* (75.0% in both modes) is the hardest domain: source pages for software-version claims are frequently dynamic or behind authentication walls, limiting the fetch benefit.

12 Discussion

12.1 Cold-Start and Network Effects

The primary risk at launch is the cold-start problem: a KB with no content attracts no agents; no agents means no contributions. The recommended launch path addresses this by (1) seeding a focused domain (Python package security facts) before opening to contributors, (2) running 2–3 independent nodes from day one, and (3) designating a cohort of trusted reviewer agents to bootstrap the reputation system.

12.2 DID Issuance and Trust Bootstrapping

AKP DIDs are self-sovereign: a DID is an Ed25519 key pair generated locally, with no registration authority, no certificate chain, and no issuance cost. This design choice eliminates single points of failure and issuance gatekeeping, but defers the question of *who should be trusted* to the governance and reputation layers.

Trust bootstrapping proceeds in three stages:

1. **Genesis**: the operator running the first node creates the initial set of KUs and seeds the reputation table with a non-zero starting score for designated reviewer DIDs via `akp.reputation.seed` (a privileged governance action requiring `rule_change` quorum).
2. **Graduation**: unregistered DIDs accumulate reviews; once `graduationThreshold` is reached, weight becomes non-zero. New entrants earn influence by correct reviewing, not by assertion.
3. **Steady state**: governance proposals modify parameters; the immutability of `graduationThreshold` via `parameter_change` prevents a graduated majority from lowering the barrier for future Sybils.

The trust model is therefore: *no trust at issuance, earned trust through verifiable behaviour, protected by cryptographic commit-reveal*. It is weaker than a PKI-based model (no DID revocation registry, no cross-network DID resolution) and stronger than an account-based model (no central issuer that can be compromised or censored).

12.3 LLM Dependency and Graceful Degradation

Stage 3 corroboration is *optional*. When no LLM backend is configured, the pipeline runs on stages 1 and 2 only, setting $S = s_1$. With $w_s = 0.40$, this reduces maximum achievable confidence by approximately 0.20 for well-sourced KUs but does not prevent the system from reaching `validated` or `stable` maturity through accumulated reviews.

Provider dependency is mitigated by the backend abstraction layer: operators choose local inference (Jan, llama.cpp — zero API cost, offline-capable) or cloud APIs (Claude, OpenAI, Gemini, OpenRouter) and can switch without changing application code. The sensitivity analysis (§11.3) shows that model capability matters more than provider: a 131B model at 93% accuracy provides genuine corroboration, while an 8B model at 53% on hard claims may degrade verdict quality. The `minCalibrationAccuracy` governance parameter (§3.4) enforces this: `runPipeline()` now runs the 20-claim calibration battery against every Stage 3 agent and filters out those that score below the threshold before their verdicts are counted. Since `minCalibrationAccuracy` is in `IMMUTABLE_VIA_PARAMETER_CHANGE`, no governance majority can subsequently lower the accuracy floor without a full `rule_change` supermajority vote.

E5's 44.4% staleness accuracy was caused by a prompt gap: the stage 3 prompt did not include the claim's submission date or `validUntil` value. The prompt now injects both, along

with today's date, so the LLM can recognise expired temporal claims without domain-specific heuristics. Re-evaluation with a live model is required to confirm the improvement.

12.4 Scalability Bounds and Known Gaps

Storage: SQLite is a single-writer database. Each AKP node maintains its own SQLite file; there is no shared-write bottleneck. Write throughput is bounded by SQLite WAL mode performance (∼10 k writes/s on NVMe). At 1 KiB per Automerge snapshot and one write per KU update, a node handling 1 000 KU updates/min consumes ≈60 MB/min of WAL before checkpoint. This is manageable for the anticipated prototype workload.

Graph traversal: BFS median latency increases from 0.02 ms to 0.04 ms between 1k and 50k nodes (Table in §10.2), confirming effectively $O(1)$ traversal in sparse graphs. Heap usage grows at 1.7 MB/1k nodes in the in-memory implementation; the production SQLite graph eliminates this scaling constraint.

Network: Automerge is AP under the CAP theorem — nodes operate independently during network partitions and converge on reconnect without coordination. Gossip convergence has been benchmarked at all peer counts up to 50 across three latency tiers (Table 11.2); all configurations converge in one round. What remains unevaluated: throughput under concurrent peer writes to the same KU, and behaviour during long-partition remerge of large delta backlogs. These are priorities for the v0.2 evaluation.

12.5 Trust and Governance Legitimacy

AKP's governance is only as legitimate as its early parameter choices. `graduationThreshold` and `proposalReputationBond` are immutable via `parameter_change` (requiring a `rule_change` supermajority), protecting the Sybil-resistance mechanism from capture by a coordinated attacker who accumulates early voting weight.

12.6 Limitations and Future Work

- **Bootstrap registry not yet deployed**: the two-level discovery layer (§6.4) and registry server (`src/sync/registry.ts`, `npm run registry`) are implemented. Nodes in production still require manual `syncPeers` configuration until a community-hosted registry endpoint is stood up.
- **Single-model ground truth**: Table 3 reports external ground truth results for Nemotron-4B only; full per-model figures across five model families remain to be evaluated.
- **Throughput and large-partition recovery**: write throughput beyond a single node and behaviour during long-partition re-merge of large delta backlogs remain unmeasured. Gossip convergence under geographic latency is characterised for up to 20 peers (Table 11.2); 50-peer intercontinental scenarios exceed the current test harness timeout.
- **Governance quorum**: minimum 5 distinct DIDs for `parameter_change` cannot be met by a single-operator deployment without development-mode overrides.
- **E5 staleness accuracy**: the stage 3 prompt now injects the KU creation date, today's date, and a note that expired `validUntil` claims should be disputed. The corrected prompt addresses the root cause identified in the original 44.4% result; re-evaluation with a live LLM backend is required to confirm the improvement.

13 Related Work

13.1 Comparative Positioning

Table 4 compares AKP against the most relevant prior systems across six dimensions relevant to agent-native knowledge infrastructure.

13.2 Centralized Knowledge Bases

Wikipedia and Wikidata [7] are the closest analogues in terms of structured, reviewed knowledge. Wikidata's property- value claim model is semantically similar to AKP's typed-claim structure. The key differences are governance (human editorial consensus vs. AKP's cryptographic commit-reveal), access (HTTP/SPARQL vs. JSON-RPC/MCP), Sybil resistance (account registration vs. DID + reputation), and confidence quantification (Wikidata has none). AKP's design is deliberately influenced by Wikidata's claim schema while targeting machine agents rather than human contributors.

13.3 Decentralised Data Protocols

Ocean Protocol [8] tokenises data assets on a blockchain, enabling access control and provenance tracking for datasets. It does not provide structured claim semantics, peer review, or confidence scoring; it solves data *access* rather than knowledge *validity*.

Ceramic Network [9] provides decentralised mutable documents over IPFS using DID-authenticated streams. It solves the same storage layer problem AKP uses (CRDTs + DIDs) but provides no application layer for claims, reviews, reputation, or governance. AKP could be built *atop* Ceramic's transport layer if cross-chain identity resolution were required.

IPFS/Filecoin addresses content-addressed storage and data persistence. Like Ceramic, it has no knowledge semantics, review mechanism, or confidence model. AKP chose SQLite + Automerge over IPFS for lower operational complexity in the prototype phase.

13.4 Federated Learning

Federated Learning (FL) [10] distributes model training across clients without sharing raw data, aggregating gradient updates rather than explicit facts. FL and AKP are complementary: FL produces updated model weights; AKP stores and verifies explicit factual claims. An agent trained via FL could *contribute*

Table 4: Comparative analysis of knowledge infrastructure approaches. ✓ = supported; ○ = partial; × = absent.

System	Decentralised	Struct. claims	Peer review	Agent-native	Sybil resist.	Confidence
AKP (this work)	✓	✓	✓	✓	✓	✓
Wikidata [7]	×	✓	○	×	×	×
Ocean Protocol [8]	✓	×	×	○	○	×
Ceramic Network [9]	✓	×	×	○	×	×
IPFS/Filecoin	✓	×	×	×	×	×
Federated Learning [10]	○	×	×	×	○	×
RAG stores (centralised)	×	×	×	○	×	×

its learned facts to AKP for cross-agent validation. FL provides no structured claim representation, claim-level peer review, or adversarial resistance to a participant who submits corrupted gradients — Byzantine-robust FL [11] is the closest analogue to AKP's Sybil resistance, but operates at the gradient level rather than the claim level.

13.5 Agent Communication Protocols

MCP [4] (Anthropic, 2024) and **A2A** [5] (Google, 2025) define agent-to-tool and agent-to-agent communication protocols respectively. AKP implements both as connection surfaces rather than replacing them: MCP makes AKP's knowledge base accessible to any MCP- capable agent; A2A enables service discovery. Neither MCP nor A2A specifies how knowledge should be stored, verified, or governed.

13.6 CRDT Research

CRDTs as a consistency primitive for collaborative systems have been extensively studied [1, 2]. AKP applies Automerge specifically to knowledge document merging, adding a validation layer (post-merge Zod schema check) that standard CRDT frameworks do not provide.

14 Conclusion

AKP demonstrates that a decentralised, adversarially-resistant knowledge infrastructure for AI agents is tractable with current tooling. The combination of CRDT-backed storage, a calibrated three-stage confidence pipeline, cryptographic commit-reveal peer review, DID-based Sybil resistance, on-chain-style governance, and a provider-agnostic LLM backend abstraction produces a system where knowledge quality is objectively measurable and the incentive structure rewards accurate verification over bandwagon behaviour. Protocol-level version gating and network isolation provide upgrade-path guarantees analogous to TLS version negotiation.

A formal threat model quantifies attack cost under temporal gate parameters, and experiments at 10× and 20× Sybil ratios confirm that the weight cap mechanism preserves honest-agent influence even at extreme ratios. Stage 3 LLM corroboration is optional and gracefully degrades: the system reaches `validated` maturity without any LLM backend, and six interchangeable provider adapters eliminate single-provider dependency.

Compared to the closest prior systems—Wikidata (human-governed, not agent-native), Ocean Protocol (data access, no claim semantics), and Ceramic Network (CRDT storage, no review layer)—AKP is the only system that combines decentralisation, structured claims, cryptographic peer review, Sybil resistance, and machine-readable confidence scoring.

Two successive audits across 13 vulnerability classes, all remediated and regression-tested, confirm that the protocol's attack surface is well-bounded. A force-directed graph dashboard, a GitHub Pages static demo, and Claude Code agent integration lower the barrier to adoption. A two-level peer discovery layer (HTTP bootstrap registry + PEX gossip), LLM calibration gating via `minCalibrationAccuracy`, and a 100-claim external ground truth corpus for RAG baseline comparison address the primary gaps identified by reviewers. Open problems: deployment of the community bootstrap registry, throughput evaluation beyond a single node, geographic latency at 50+ peers, and staleness prompt engineering to improve E5 accuracy. 266 tests across 34 files, nine multi-agent experiments, and a fully self-contained reproducibility package support independent verification of all reported results. Source code and deployment configurations are available at https://github.com/Pataca/akp.

A Deployment Configurations

AKP ships with five pre-configured deployment paths:

- **Local**: `npm run setup && npm start` — single command, Node.js 18+ only; identity auto-generated on first run; API key auto-generated if `AKP_API_KEY` is not set.
- **Docker**: multi-stage Alpine build; non-root user; persistent `/data` volume; health check on `/health`.
- **Fly.io**: `fly.toml` pre-configured; 512 MB VM; 1 GB persistent volume; `fly launch && fly deploy`.
- **Render**: `render.yaml` auto-detected; persistent disk on Starter plan.
- **GitHub Codespaces**: `.devcontainer/` pre-configured; `npm run setup` runs on container create; port 3000 forwarded.

Network isolation for testing:

AKP_NETWORK_ID=testnet npm start. The node will refuse connections from peers advertising a different network ID.

B Human Dashboard — Interaction Details

The React SPA (ui/) communicates exclusively via the /rpc JSON-RPC endpoint. Auth is handled by reading requiresAuth from the /health response on startup; a bearer-token modal appears when authentication is required. A global akp:unauthorized DOM event re-triggers the modal on any subsequent 401 without a page reload.

The knowledge graph view (#/graph) uses a pure HTML5 Canvas force-directed layout (no external library): repulsion, spring, gravity, and damping forces run for 300 warm-up ticks before display. Node rendering: radius $\propto \sqrt{\text{confidence}}$, colour by maturity (draft=grey, proposed=yellow, validated=green, stable=purple). Edge strength: shared claim subjects (0.9), same domain (0.3). Zoom 0.15–5$\times$ via scroll wheel; drag-to-pan; click-to-toggle floating info box.

Claude Code skills (.claude/commands/ — contribute, query, review) provide AI agents with structured step-by-step workflows. CLAUDE.md at the repository root supplies onboarding context and API quick-reference to any agent that clones the repository.

References

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free Replicated Data Types. In *Proc. SSS 2011*, LNCS 6976, pp. 386–400, 2011.

[2] M. Kleppmann and G. Beresford. A Conflict-Free Replicated JSON Datatype. *IEEE Trans. Parallel Distrib. Syst.*, 28(10):2733–2746, 2017.

[3] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, July 2022.

[4] Anthropic. Model Context Protocol Specification. https://modelcontextprotocol.io, 2024.

[5] Google. Agent-to-Agent (A2A) Protocol Specification. https://google.github.io/A2A, 2025.

[6] P. Koval. @noble/ed25519: audited Ed25519 in TypeScript. https://github.com/paulmillr/noble-ed25519, 2024.

[7] D. Vrandečić and M. Krötzsch. Wikidata: A free collaborative knowledge base. *Commun. ACM*, 57(10):78–85, 2014.

[8] T. McConaghy et al. Ocean Protocol: Tools for the Web3 Data Economy. *White Paper*, Ocean Protocol Foundation, 2019. https://oceanprotocol.com/tech-whitepaper.pdf

[9] J. Rawson et al. Ceramic Protocol: A Decentralized Data Network. *Technical Specification v1.0*, 3Box Labs, 2021. https://ceramic.network

[10] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. AISTATS 2017*, pp. 1273–1282, 2017.

[11] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer. Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent. In *Proc. NeurIPS 2017*, pp. 119–129, 2017.

[12] AKP Research Group. AKP Prototype v0.1 — Known Limitations. Technical report, https://github.com/Patacka/akp/blob/master/paper/KNOWN-LIMITATIONS.md, 2026.